



MPO: 通过 Meta Plan Optimization 提升LLM代理

Weimin Xiong, 宋一凡, 董庆秀, 赵炳婵, 宋飞凡, 王迅, 李

素健

♡北京大学♣计算机科学学院 北京大学 多媒体信息处理国家重点实验室

{wmxiong, lisujian}@pku.edu.cn <https://github.com/WeiminXiong/MPO>

抽象

大型语言模型 (LLMs) 的最新进展使LLM基于智能体的智能体能够成功处理交互式规划任务。然而, 尽管取得了成功, 但现有方法经常受到计划幻觉的影响, 并且需要对每个新代理进行再培训。为了应对这些挑战, 我们提出了 Meta Plan Optimization (MPO) 框架, 该框架通过直接结合明确的指导来增强代理规划能力。与以前依赖复杂知识的方法不同, 这些方法要么需要大量的人力, 要么缺乏质量保证, 而 MPO 通过 meta 计划利用高级通用指导来协助理规划, 并根据代理任务执行的反馈持续优化 meta 计划。我们对两项代表性任务进行的实验表明, MPO 的性能明显优于现有基线。此外, 我们的分析表明, MPO 提供了一种即插即用的解决方案, 可以在以前看不见的场景中提高任务完成效率和泛化能力。

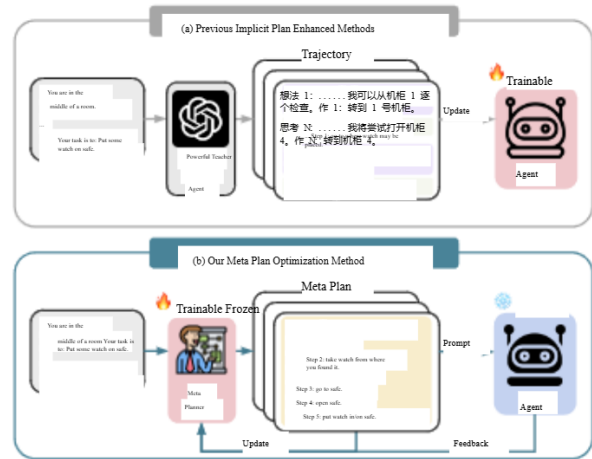


图 1: 与以前需要智能体参数更新的隐式计划增强方法不同, 我们的方法将元计划纳入直接规划指导的提示中, 并可以根据反馈对其进行改进。

1 引言

大型语言模型的最新进展 (LLMs) (Achiam et al., 2023; Liu et al., 2024; Yang et al., 2024a) 使LLM基于智能体的代理能够处理复杂的多步骤任务, 包括具身家务劳动 (Shridhar et al., 2020) 和科学实验 (Wang et al., 2022)。这些任务需要复杂的规划能力, 因为代理需要了解长期依赖关系 (Zhang et al., 2024), 推理顺序行动, 并适应动态环境 (Yao et al., 2022)。这些代理的规划质量在决定其整体绩效方面起着至关重要的作用。

目前主流LLM的代理主要通过以下方式发展他们的规划能力

隐式方法, 要么直接利用模型的内在能力, 要么根据专家轨迹进行微调。例如, ReAct (Yao et al., 2022) 和 Reflexion (Shinn et al., 2024) 在任务执行过程中即时执行规划, 并且容易因规划幻觉而迷失方向 (Zhu et al., 2024)。包括 AgentTuning (Zeng et al., 2023)、Lumos (Yin et al., 2023) 和 ETO (Song et al., 2024b) 在内的方法采用轨迹调优来增强隐式规划能力, 并且需要对每个新代理进行再训练, 从而导致巨大的计算成本 (图 1 (a))。

除了隐性规划之外, 一些研究已经开始探索使用显性知识来指导代理执行任务 (Zhu et al., 2024; Qiao 等人, 2024 年)。虽然这些工作为智能体带来了明确指导、可解释性和低集成成本等优势, 但它们要么需要大量的人工设计工作, 要么在复杂的知识获取过程中缺乏质量保证, 导致智能体的改进不一致 (Wang et al., 2024)。构建

*通讯作者。

这些努力，我们引入了 Meta Plan 的概念，它提供了高级、抽象的指导来协助代理规划。如图 1

(b) 所示，对于任务“put some watch on safe”，元计划概述了完成一般任务的抽象策略。与之前在执行过程中获得的隐式计划不同，元计划与特定的环境细节（例如“cabinet 1”、“cabinet 4”）和复杂的智能体轨迹解耦，使其更适合优化。

此外，为了自动提高元计划的质量，我们提出了一个优化框架，即元计划优化（MPO），它由一个元规划器和一个代理组成。元规划师负责生成高级元计划，而代理则提供有关执行的反馈，以评估输入的元计划的质量并帮助完善元规划器。最初，我们从专家轨迹中收集元计划，并通过监督微调对元规划器进行冷启动。为了进一步优化元规划器，我们使用蒙特卡洛（MC）抽样来估计智能体的任务完成率作为反馈。具体来说，给定一个任务，规划者通过抽样生成多个元计划。然后对于每个元计划，也对 agent 进行采样，产生多个执行轨迹，并据此估算任务完成率。在确定了对比性的元计划对（导致最高和最低任务完成率的那些）之后，我们应用 DPO (Rafailov 等人, 2024 年) 来改进这些计划对的元规划器。最后，训练有素的 meta planner 可以从 MPO 框架中分离出来，作为一个即插即用的组件，能够为目标环境中的任务生成高质量的 meta plan。这有助于完成任何新代理的任务，而不会产生额外的培训成本。

我们根据两个具有代表性的基准评估我们的方法：用于具体家务任务的 ALFWorld (Shridhar et al., 2020) 和用于文本科学实验任务的 ScienceWorld (Wang et al., 2022)。在所有测试任务中，配备 Meta Planner 的代理的表现明显优于没有它的代理，性能提升高达 100%。此外，meta planner 还与各种现有的代理框架兼容。当与这些方法结合使用时，out 方法可以提供更好的性能提升，这证明了它在更大的应用范围内的有效性

进一步分析显示，我们生成的元计划显著提高了智能体每次作的平均奖励，从而提高了任务完成效率。

总而言之，我们的贡献如下：

- 我们介绍了 MPO，它利用元计划优化来提高代理的性能 LLM。这一进展提供了一种创新方法，以即插即用的方式增强代理的规划能力，同时保持与现有框架的兼容性。
- 对两个具有代表性的基准进行的广泛实验表明，我们的方法显著提高了现有 LLM 代理的性能。
- 进一步分析表明：(1) 我们提出的方法大大提高了智能体的任务完成效率；(2) 轻量级的 meta planner 可以指导更强大的代理人进行规划。(3) MPO 提高了元计划的正确性、可遵循性和标准化

2 任务制定

LLM 智能体规划 本研究的主要范围是规划 LLM 智能体与环境交互并接收任务解决方案的反馈。根据 Song et al. (2024b)，智能体的任务规划轨迹可以表示为 $e = (u, a, o, \dots, a)$ ，其中 $u \in U$ 是任务指令， $a \in A$ 是智能体行动， $o \in O$ 表示对环境的观察。在每个时间步 t 中，代理执行隐式规划并生成相应的作

$a \sim \pi(\cdot | u, a, o, \dots, o)$ 。生成任务规划轨迹的概率由下式给出：

$$\pi(e|u) = \prod_{t=1}^T \pi(a_t | u, a, o, \dots, o) \quad (1)$$

最后，计算代表任务完成率的最终奖励 $r(u, e) \in [0, 1]$ 。

元计划 元计划作为高级、自然的指导，以协助代理规划。它概述了一种抽象的、通用的任务完成策略，该策略与特定的环境细节分离，表明它有可能在各种代理中泛化。例如，给定指令“look at the CD under the desk lamp”，元计划可以是：“1. 去可以放置 CD 的地方。2. 从您找到它的地方拿 CD。3. 前往台灯所在的位置。4. 用途

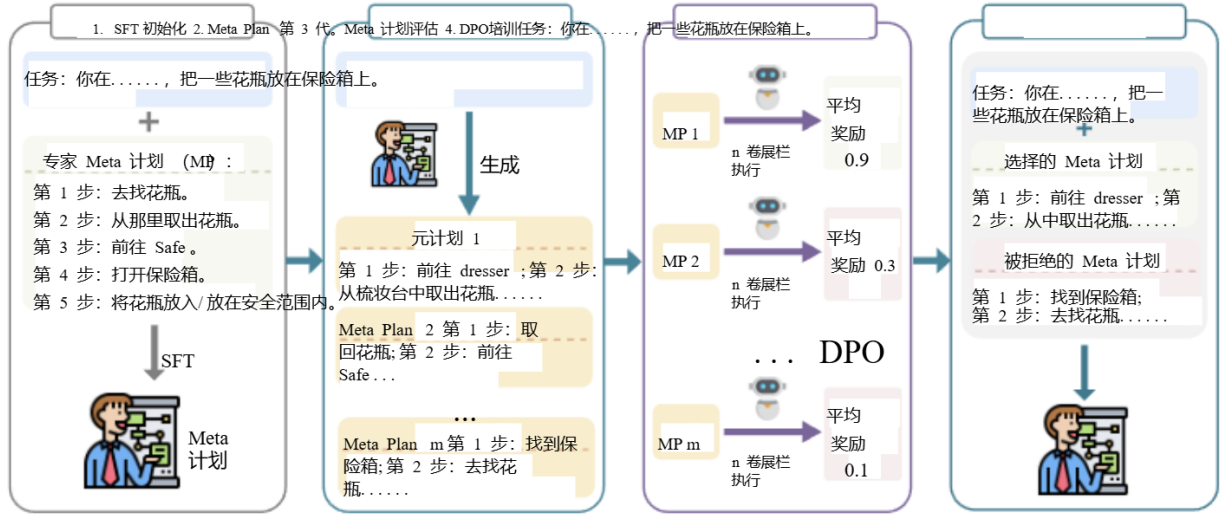


图 2: MPO 的整体架构。元规划器首先在种子元计划 (MP) 集上进行监督微调。然后, 我们通过对对比元计划的偏好学习来优化元规划器。

看 CD 的台灯。低质量的元计划可能会误导代理的规划过程。为了保证元计划的质量, MPO 开发了一个轻量级参数化的元计划器 π 来生成元计划, 可以进一步优化以产生更好的结果。在纳入元计划后, 智能体产生轨迹 e 的概率表示为:

$$\pi(e|u, p) = \prod_{t=1}^T \pi(a_t|u, p, a_{1:t-1}, o_{1:t-1}) \pi(p|u) \quad (2)$$

3 方法

我们方法的整体框架如图 2 所示。首先, 我们构建一个种子元计划训练集来初始化一个基本的元规划器 (§ 3.1)。然后, 我们开发了 MC 方法, 通过探索来评估元计划的质量 (§ 3.2)。最后, 我们通过使用对比元计划对 (§ 3.3) 进行基于偏好的优化, 进一步增强了元规划器。

3.1 监督微调初始化

为了使元规划器具备根据任务指令和环境状态生成元计划的基础能力, 我们使用监督微调来初始化模型。但是, 现有的智能体数据集只提供了黄金任务完成轨迹, 没有相应的元计划。因此, 我们首先需要构建一个训练数据集来生成元计划。为此, 我们利用 GPT-4o 来协助创建数据集

我们以原始任务指令 u 和对应的黄金轨迹 e 作为提示, 为模型提供原始任务指令 u , 并允许模型从轨迹中总结出一个可泛化的计划 p 。具体的提示模板可以在附录 E.1 中找到。为了确保元计划 p 的质量, 我们手动审查 GPT-4o 生成的结果并改进任何不正确、过于复杂或不标准的元计划。这个质量控制过程确保每个元计划 p 代表一个可重用的规划策略, 有效地帮助代理完成任务。控制种子元计划集质量的详细流程可以在附录 C 中找到。由于 meta planner 在推理过程中需要生成无法访问黄金轨迹的计划, 因此我们将它们从训练数据中删除, 从而获得 meta planner 的初始化数据集:

$$D = \{(u, p)\}_{i=1}^n \quad (3)$$

然后, 我们在自回归损失上微调模型, 以获得初始化的元规划器 π :

$$L = -\mathbb{E}[\text{对数 } \pi(p|u)] \quad (4)$$

3.2 元计划质量评估

为了进一步增强 meta planner, 我们需要评估其生成的 meta plans 的质量。虽然以前的研究通常依赖于在人类偏好注释上训练的奖励模型 (Bai et al., 2022a; Ouyang et al., 2022; Dubey 等人, 2024 年) 或高级人工智能 (Bai et al., 2022b; Lee et al., 2023) 模型来评估模型输出, 这些方法都有局限性 它们不仅会产生额外的

人工标记或 API 调用的成本，但也可能不太适用于 LLM 代理，因为他们对 Meta 计划的偏好与代理或任务环境不一致。为了规避这些挑战，我们采用基于探索的方法来评估元计划的质量。

直观地说，更高质量的元计划应该使代理更容易成功完成任务。因此，对于一个 give meta plan p ，我们将其插入到代理的提示符中，并让代理尝试完成任务 N 次。这会导致代理生成 N 个任务完成轨迹：

$$\{e_i | i = 1, \dots, N\} \sim \pi(e|u, p) \quad (5)$$

对于每个轨迹 e ，环境返回任务完成率 $r(u, e)$ 。因此，元计划 p 的质量是由基于它的智能体完成任务的成功率决定的，可以表示为：

$$Q(p) = \frac{1}{N} \sum_{i=1}^N r(u, e_i) \quad (6)$$

在本文中，我们使用 Llama-3.1-8B-Instruct (Dubey et al., 2024) 作为代理来评估元计划的质量。此模型展示了强大的指示-跟随能力，并且已经有效地完成了代理任务。此外，用这个模型评估的元计划可以推广到基于其他模型的代理，我们将在后面的实验中验证。

3.3 Meta Planner DPO 培训

在我们能够自动评估元计划的质量后，我们可以通过强化学习进一步优化 SFT 初始化的元规划器。我们选择 DPO (Rafailov et al., 2024) 作为我们的优化算法，因为它的训练稳定性和低资源消耗。DPO 算法需要配对的首选数据来优化元规划器，特别是高质量和低质量的元计划对。我们从任务训练集中构建 DPO 偏好数据集 D ，其中 SFT 初始化的元规划器生成 M 个元计划 $\{p_i | i = 1, \dots, M\} \sim \pi(p|u)$ 。然后，我们使用第 3.2 节中描述的 MC 方法计算每个元计划的分数。选择最高和最低质量的元计划作为选择和拒绝的 p_{and} 和 p_{ref} 对。如果所有元计划的质量相同，我们将跳过此示例。这构成了我们的偏好训练数据集：

$$D = \{(u, p_{\text{and}}, p_{\text{ref}})\}_{i=1}^M \quad (7)$$

数据集 训练 测试 已见测试 未见过的作空间

科学世界 1483 194 211 19 ALFWorld 3321 140 134 13

表 1: 测试数据集的统计概述。“Test Seen” 和 “Test Unseen” 分别是具有已见和未见场景的测试集。

给定偏好数据集 D ，DPO 优化模型以增加所选元计划覆盖被拒绝的 p_{ref} 的可能性。我们通过最小化 DPO 损失来微调 meta planner：

$$\mathcal{L}(\pi; \pi_{\text{ref}}) = -\mathbb{E}_{(u, p_{\text{and}}, p_{\text{ref}}) \sim D} \left[\log \sigma(\beta \log \frac{\pi(p_{\text{and}}|u)}{\pi(p_{\text{ref}}|u)}) - \beta \log \frac{\pi(p_{\text{ref}}|u)}{\pi(p_{\text{and}}|u)} \right] \quad (8)$$

(8) 这个方程反映了为给定的任务指令 u 最大化生成高质量元计划 p_{and} 而非较低质量元计划 p_{ref} 的概率的目标。通过构建偏好数据集并应用 DPO 优化，元规划器可以更有效地生成高质量的元计划，从而更好地指导代理规划过程。

4 实验

4.1 实验设置

数据集 我们在两个具有代表性的代理数据集上进行了实验：用于文本科学实验任务的 ScienceWorld (Wang et al., 2022) 和用于具体家庭任务的 ALFWorld (Shridhar et al., 2020)。ScienceWorld 提供从 0 到 1 的密集最终奖励，而 ALFWorld 仅提供二进制奖励，表明任务是否已完成。有关数据集的详细信息，请参阅附录 A。我们数据集的统计信息如表 1 所示。需要注意的是，除了分布内测试集之外，ALFWorld 和 ScienceWorld 都包含包含分布外看不见的变化的测试集。这些额外的测试集使我们能够评估 meta planner 的泛化能力。

实现细节 我们使用 Llama-3.1-8B-

Instruct (Dubey et al., 2024) 作为构建元规划器的基本模型。对于 SFT 初始化，我们将批量大小设置为 32，将学习率设置为 $1e-5$ ，并使用具有 3 个训练 epoch 的余弦调度器。对于 DPO (Rafailov et al., 2024) 训练，我们将元规划器配置为生成 $M = 5$

型号无 Exp.ScienceWorld ALFWorld 平均 看到 未见 看到 不可见									
未接受培训的座席									
GPT-4o (Achiam 等人, 2023 年) X 60.0 56.0 78.6 83.6 69.6 GPT-4o-mini (Achiam 等人, 2023 年) X 49.1 42.7 32.1 41.0 41.2 Llama-3.1-8B-Instruct (Dubey 等人, 2024) X 47.7 42.2 22.9 28.4 35.3 Qwen2.5-7B-Instruct (Yang et al., 2024a) X 38.5 38.8 71.4 75.4 56.0 Llama-3.1-70B-Instruct (Dubey et al., 2024) X 72.6 70.2 78.6 73.9 73.8 Llama-3.1-8B-Instruct + MPO ✓ 56.5 55.5 50.0 52.2 53.6 GPT-4o + MPO ✓ 67.3 67.8 89.3 93.3 79.4 Llama-3.1-70B-Instruct + MPO ✓ 80.479.5 85.7 86.6 83.1									
受过培训的代理									
Llama-3.1-8B-Instruct + SFT (Zeng 等人, 2023 年) X 65.3 57.0 79.3 71.6 68.3 Llama-3.1-8B-Instruct + ETO (Song 等人, 2024b) X 81.3 74.1 77.1 76.4 77.2 骆驼-3.1-8B-Instruct + KnowAgent (Zhu et al., 2024) ✓ 81.7 69.6 80.0 74.9 76.6 骆驼-3.1-8B-Instruct + WKM (Qiao et al., 2024) ✓ 82.1 76.5 77.1 78.2 78.5 骆驼-3.1-8B-Instruct - SFT+ MPO ✓ 70.2 65.9 80.7 81.3 74.5 骆驼-3.1-8B-Instruct -ETO+ MPO ✓ 83.4 80.8 85.0 79.1 82.1									

表 2：不同方法在两个数据集上的性能。MPO 优化的元计划显著提高了各种模型或代理框架的性能，超越了其他显式指导（Exp. Guid.）方法。

每个任务的 meta 计划，生成温度为 0.7。为了评估元计划的质量，我们将代理设置为为每个元计划生成 $N = 5$ 个任务完成轨迹，也使用 0.7 的温度。我们利用 vLLM (Kwon et al., 2023) 来加速生成过程。对于 DPO 训练，批量大小为 32，学习率为 $1e-5$ ，预热阶段为 3%，并使用余弦调度程序。对于 ALFWorld 和 ScienceWorld 数据集，DPO 损失函数中的 β 参数都设置为 0.1，训练在 3 个时期进行。所有训练程序均使用 Llama-Factory (Zheng et al., 2024) 实施，并进行完整的参数微调。实验在 8 个 NVIDIA A100 80GB GPU 上进行。

基础代理 我们在 MPO 优化的元计划指导下，在两种类型的代理上评估我们的方法：（1）未经培训的代理，使用基础模型部署 ReAct 框架，无需额外培训。我们测试了两个专有模型，包括 GPT-4o 和 GPT-4omini (Achiam 等人, 2023 年) 以及几个开源模型，包括 Llama-3.1-8B-Instruct、Llama-3.1-70B-Instruct (Dubey 等人, 2024 年) 和 Qwen2.5-7B-Instruct (Yang 等人, 2024a)。（2）经过培训的智能体，通过对基础模型的参数更新来增强智能体规划能力。我们研究了两个代理框架：AgentTuning (Zeng et al., 2023)

它使用来自专家轨迹的监督微调来提高基础模型的代理能力，以及 ETO (Song et al., 2024b)，它从失败的轨迹中学习并提出一种基于探索的轨迹优化方法来增强任务解决过程。我们还与 KnowAgent (Zhu et al., 2024) 和 WKM (Qiao et al., 2024) 进行了比较，它们也为代理规划过程注入了明确的指导。这两种方法需要微调其基础模型，使其与其他代理框架不兼容。

评估 为了确保实验的可重复性，我们将元规划器生成的元计划和代理生成的任务轨迹的解码温度设置为 0。对于元计划的生成，我们采用零样本提示方法。在生成任务完成轨迹时，我们为每个任务提供了一个 1-shot 上下文示例。详细提示见附录 E2。请注意，一旦 meta planner 生成了测试集任务的元计划，我们就会在所有代理中使用它们，而无需进一步修改。我们的主要评估指标是 Average Reward，它计算所有测试集任务实例的平均奖励。我们还在附录 B 中报告了成功率。我们将在接受后发布优化的 meta planner 生成的 meta 计划和参数。

基础LLM设置	SciWorld	ALFWorld
GPT-4o 机器人	- 56.0 83.6 接线端子SFT 59.5 91.0 RFT 61.8 89.6 城市规划项目 (MPO) 67.8 93.3	
美洲驼-3.1-8B-INS	- 42.2 28.4 接线端子SFT 45.6 43.3 RFT 50.5 47.8 城市布线 (MPO) 55.5 52.2	
Qwen2.5-7B-Ins	- 38.8 75.4 接线端子SFT 37.4 73.9 RFT 41.9 78.3 城市布线 (MPO) 43.7 82.8	

表 3: meta planner 优化方法的消融研究。

4.2 结果

如表 2 所示, MPOOptimized 元计划的加入持续提高了所有任务和框架的代理性能, 基于 Llama-3.1-8B-Instruct 的代理的平均性能提高了 51.8%。此外, 我们的 meta planner 与其他代理培训框架兼容。MPO 增强的 Llama-3.1-8B-Instruct -ETO 的平均奖励比当前的 SOTA 显式引导方法 WKM 高 3.6 倍。这些结果表明, 我们通过代理反馈优化的通用高级元计划优于基于知识的复杂指导, 这些指导严重依赖人工工作, 缺乏泛化能力, 并且不提供质量保证。这些结果突出了我们的方法在提高代理性能方面的有效性。此外, 我们的方法在看不见的场景中表现出很强的有效性。对于 ScienceWorld 和 ALFWorld 中看不见的部分, 尽管从未遇到过这些任务, 但 meta planner 能够泛化到它们并生成高质量的 meta 计划。这将 GPT-4o 在 ALFWorld 的看不见部分的成功率提高了 11.1, 达到了 93.3 的成功率。这些结果强调, MPO 可以进一步增强代理的泛化能力, 尤其是在分布外场景中。

5 分析

5.1 消融研究

我们对 meta planner 的训练方法进行了消融实验。对于 ScienceWorld 和 ALFWorld, 我们在不同的测试集上进行评估

基本LLM类型	SciWorld	ALFWorld
GPT-4o 机器人	研究所 67.8 93.3 你。65.3 85.1 观察 67.6 91.8	
美洲驼-3.1-8B	机构 55.5 52.2 你。38.0 34.3 观察 53.3 50.8	
骆驼-3.1-8B-SFT	研究所 65.9 81.3 你。47.9 25.4 观察 60.6 67.2	

表 4: 不同元计划插入位置对代理绩效的影响。

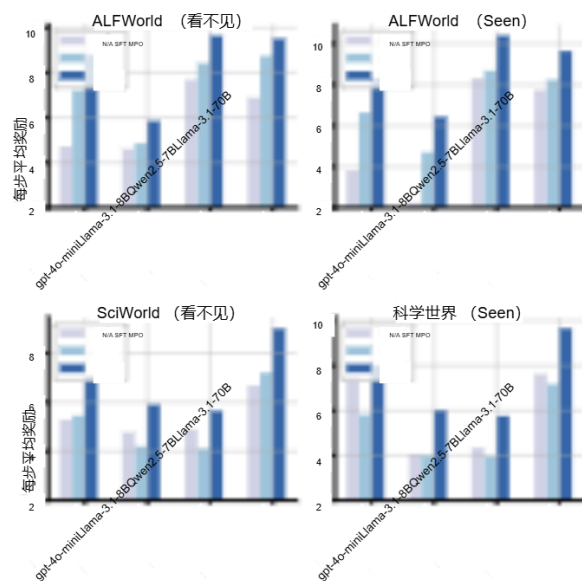


图 3: 每一步的平均奖励。

如表 3 所示, 与其他训练方法相比, MPO 优化的元规划器可以更大程度地提高智能体性能, 这表明探索环境和从比较中学习有助于元规划器生成更高质量的元计划。此外, 当使用 SFT 初始化的元计划时, Qwen2.5-7B-Instruct 模型在两个评估数据集上的性能都会下降, 这表明低质量的元计划可能会误导智能体规划过程。

5.2 如何使用 Meta 计划?

在我们的主要实验中, 元计划被纳入任务说明中, 以指导代理规划过程。在这里, 我们研究了不同插入位置对智能体性能的影响: 在任务指令中, 在智能体的思维过程中, 在环境观察中。如表 4 所示, 我们发现插入到

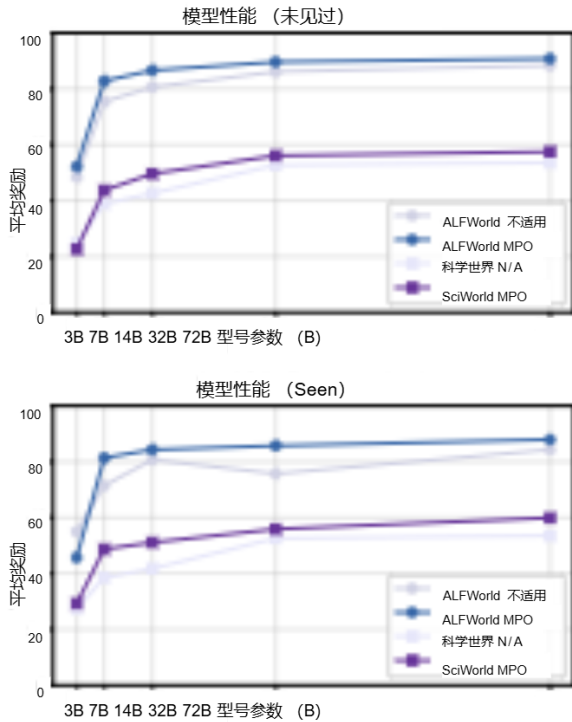


图 4：MPO 在不同参数大小的代理中的有效性。

任务指令在所有代理和任务中始终产生最佳性能，而插入 Thinking Process 会导致最差的性能。这表明破坏代理的正常推理过程会对计划的准确性产生负面影响。此外，我们观察到，将元计划插入其他位置会导致训练的智能体框架的性能下降更大，这可能是因为训练数据不涉及元计划。相反，插入指令对原始任务完成过程造成的干扰最小。这些结果表明，在任务指令中插入元计划可确保最佳性能。

5.3 效率分析

纳入高质量元计划的另一个好处是，它可以防止代理进行不必要的探索，从而提高他们的任务完成率。根据 Xiong 等人 (2024 年)，我们使用每步的平均奖励来评估行动效率，为每个任务计算为最终奖励与完成任务所需的步骤数的比率，然后在整个测试集中平均这些值。

图 3 显示了与 nometa 计划 (N/A) 和 SFT 初始化的 meta 计划相比，我们的 MPO 实现的平均步长奖励的显著改进。同样明显的是，对于看不见的测试任务，MPO 导致每步平均奖励的增幅更大，证明了它对分布外任务的强烈泛化。这些结果强调了 MPO 的卓越性能，证实了其在提高代理动作效率方面的有效性。

5.4 对具有缩放参数的代理的影响

为了进一步验证我们方法的有效性，我们对具有不同参数大小的模型进行了实验。我们选择 Qwen2.5-Instruct 系列作为测试模型，选择从小到大的一系列参数大小：3B、7B、14B、32B 和 72B，并评估它们在 ScienceWorld 和 ALFWorld 的可见和不可见部分的性能。MPO 对具有不同参数大小的代理的有效性如图 4 所示。我们观察到，随着代理参数大小的增加，MPO 的性能改进最初增加，然后减少。这可能是因为 72B 车型已经具备了很强的规划能力，使得 MPO 的改进相对有限。此外，对于 3B 模型，由于其 instructionfollow 能力有限，该模型难以有效利用元计划。因此，将 meta plan 插入提示实际上会导致性能下降。这些结果表明，MPO 可以提高各种参数大小的智能体性能，在中等参数的智能体中观察到的改进最为显著。此外，作为一个轻量级模型，元规划器有可能以即插即用的方式增强更强大的代理，而无需重新训练代理。

5.5 什么是好的 Meta 计划？

我们进一步研究了为什么通过在 MPO 中探索优化的元计划优于通过 SFT 初始化获得的元计划。我们从正确性、可跟踪性和标准化三个角度评估元计划，使用 GPT-4o 进行自动评估。评估详情和提示可在附录 E3 中找到。如图 5 所示，MPO 优化的元计划在所有三个维度上都始终优于 SFT 初始化的元计划。正确性和可跟踪性方面的优势使代理更容易有效地计划和执行任务，从而提高任务完成率。有关更详细的案例研究，请参阅附录 D。

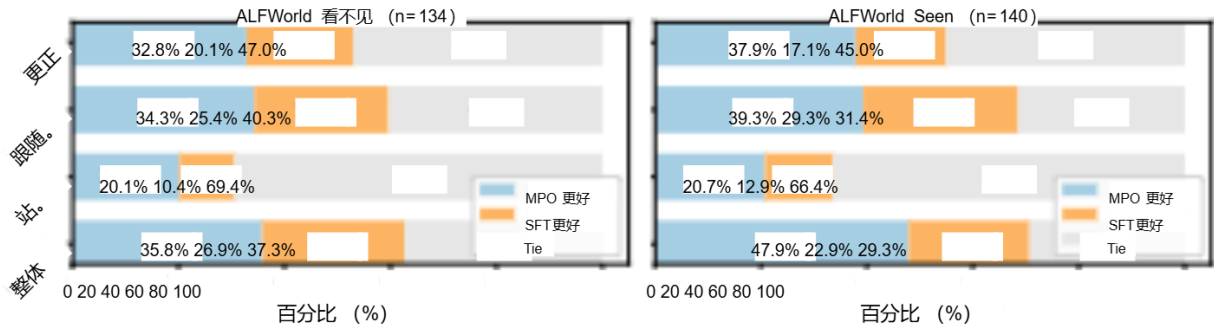


图 5: ALFWorld 上 SFT初始化和 MPO 优化的元计划的比较。

方法	Plug-and-Play	Explicit	指导	环境的适应	人力
反应	✓	×	×	×	✓
代理调整	×	×	×	✓	ETD
	×	×	×	×	KnowAgent
	×	×	×	×	WKM
	×	×	×	×	✓
	×	×	×	×	✓
MPO	✓	✓	✓	✓	✓

表 5: MPO 与替代方案: MPO 以即插即用的方式为代理规划提供明确的指导, 并利用环境反馈进行优化。

6 相关工作

LLM作为代理 随着大型语言模型的推理和指令跟踪能力的进步 (Wei et al., 2022a), 研究人员已经开始使用提示方法 (Wei et al., 2022b; Song et al., 2023) 或更复杂的策略 (Koh et al., 2024) 来构建能够利用工具 (Qin et al., 2023)、解决问题、编写代码 (Qian et al., 2023) 和完成现实世界任务的代理 (Patil et al., 2023; Gur 等人, 2023 年; Yang et al., 2024b)。为了增强开源模型作为代理的能力, 一些工作 (Zeng et al., 2023; Song et al., 2024a) 已经开始使用专家轨迹进行监督微调 LLMs, 而其他 (Song et al., 2024b; Xiong et al., 2024) 使代理能够自主探索环境, 并利用强化学习从失败的经历中学习。但是, 每次部署新代理时, 这些方法都需要重新训练, 这会导致大量的计算开销。

代理规划中的LLM规划 (Huang et al., 2024) 对于智能代理完成现实世界的任务至关重要, 涉及将复杂的指令分解为子任务并按顺序执行它们。以前的工作 (Yao et al., 2022; Shinn et al., 2024) 主要关注隐性规划, 其中规划是通过交错推理和行动生成来实现的

为了解决内隐规划中近视推理和规划幻觉的挑战 (Zhu et al., 2024), 一些方法 (Guan et al., 2024; Li et al., 2024; Zhao et al., 2024; Zhu et al., 2024) 探索了使用显性知识来指导任务执行。但是, 这些方法通常需要手动设计的提示模板或任务过程, 因此很难在不同环境中传输。一些作品 (周 et al., 2023; Ye et al., 2023; Fu et al., 2024) 使用语言模型来自动化任务知识合成, 但生成的知识无法通过探索和环境反馈进一步优化, 导致性能欠佳。相比之下, 我们的 MPO 引入了一个自动生成的元计划, 该计划提供高级、抽象的指导来协助代理规划, 同时还允许根据代理任务完成过程的反馈进一步提高质量。

将 MPO 与表 5 中的几种替代方案进行比较, 突出了我们的方法在增强LLM代理规划能力方面的优势。

7 总结

在本文中, 我们提出了 MPO, 这是一种用于增强基于 LLM 的代理的规划能力的新框架。MPO 通过 Meta Plans 集成抽象的高级指导, 提供即插即用的解决方案, 以有效提高代理绩效。通过利用代理任务执行的反馈, MPO 可以不断提高元计划的质量。对两个基准的广泛实验表明, 我们的框架始终优于现有基线, 并且适用于各种参数大小的代理。这些发现突出了我们方法在提高代理规划能力方面的潜力, 为通用人工智能的未来发展铺平了道路。

2022. 训练语言模型以遵循人类反馈的说明。神经信息处理系统的进展, 35: 27730–27744。

Shishir G Patil, Tianjun Zhang, Xin Wang 和 Joseph E Gonzalez. 2023. Gorilla: 与海量 API 连接的大型语言模型。arXiv 预印本 arXiv: 2305.15334。

陈倩、丛欣、程阳、陈伟泽、苏玉生、徐菊元、刘志远和孙茂松。2023 年。用于软件开发的通信代理。arXiv 预印本 arXiv: 2307.07924, 6 (3)。

乔硕飞、方润楠、张宁宇、朱玉琦、陈翔、邓淑敏、江勇、谢鹏军、黄飞和陈华军。2024. 使用世界知识模型进行代理规划。CoRR, abs/2405.14205。

秦玉佳、梁世豪、叶一宁、朱昆仑、闫蓝、卢亚曦、林燕凯、丛欣、唐祥如、钱磊等人, 2023 年。TooIIm: 帮助大型语言模型掌握 16000+ 个真实世界的 API。

arXiv 预印本 arXiv: 2307.16789。

拉斐尔·拉法洛夫、阿奇特·夏尔马、埃里克·米切尔、克里斯托弗·曼宁、斯特凡诺·埃尔蒙和切尔西·芬恩。

2024. 直接偏好优化: 你的语言模型秘密地是一个奖励模型。神经信息处理系统的进展, 36。

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan 和 Shunyu Yao. 2024. 反思: 具

体智能的反思式学习。神经信息处理系统的进展, 36。

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler 和 Matthew Hausknecht. 2020. Alfworld: 对齐文本和具体环境以进行交互式学习。arXiv 预印本 arXiv: 2010.03768。

宋一帆、熊伟民、赵秀田、朱大伟、吴文浩、王可、李成、彭伟和李素健。2024a. 代理库: 通过对 50000+ 交互轨迹进行微调, 迈向通用 IIm 代理。arXiv 预印本 arXiv: 2410.07706。

宋一凡、熊伟民、朱大伟、吴文浩、韩倩、宋明波、黄海亮、李成、王珂、姚荣等人, 2023 年。Restgpt: 将大型语言模型与现实世界的 restful api 连接起来。arXiv 预印本 arXiv: 2306.06624。

宋一凡、尹大、岳翔、黄杰、李素健和林玉辰。2024b. 试错法: 基于探索的 IIm 智能体轨迹优化。arXiv 预印本 arXiv: 2403.02502。

Fei Wang, Xingchen Wan, Ruoxi Sun, Jiefeng Chen, 和 Serkan Ö. Arik. 2024. Astute rag: 克服大型语言模型的不完美检索增强和知识冲突。预印本, arXiv: 2410.07176。

Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté 和 Prithviraj Ammanabrolu。2022. 科学世界: 你的经纪人比五年级学生更聪明吗? arXiv 预印本 arXiv: 2203.07540。

Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani

2022 年a. 大型语言模型的 emergent abilities。arXiv 预印本 arXiv: 2206.07682。

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny 周 et al. 2022b. 思维链提示在大型语言模型中引发推理。神经信息处理系统的进展, 35: 24824–24837。

Weimin Xiong, Yifan Song, Xiutian Zhao, Wenhao Wu, Xun Wang, Ke Wang, Cheng Li, Wei Peng, and Sujian Li. 2024. 注意每一步! IIm 通过迭代步骤级流程优化进行代理学习。arXiv 预印本 arXiv: 2406.11176。

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024a. Qwen2.5 技术报告。arXiv 预印本 arXiv: 2412.15115。

John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan 和 Ofir Press. 2024b. Swe-agent: 代理-计算机接口支持自动化软件工程。arXiv 预印本 arXiv: 2405.15793。

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan 和 Yuan Cao. 2024. React: 在语言模型中协同推理和作。arXiv 预印本 arXiv: 2210.03629。

叶一宁、丛欣、田静zuo、曹剑南、王浩、秦玉佳、卢亚曦、俞合阳、王华东、林燕凯等人, 2023 年。Proagent: 从机器人流程自动化到代理流程自动化。arXiv 预印本 arXiv: 2311.10751。

Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi 和 Bill Yuchen Lin. 2023 年。Lumos: 具有统一数据、模块化设计和开源的学习代理

llms。在 ICLR2024 大型语言模型 () LLM 代理研讨会中。

Aohan Zeng, Mingdao Liu, Rui Lu, Bowen Wang, Xiao Liu, Yuxiao Dong, 和 Jie Tang. 2023. 代理调整: 为 llms 启用通用代理功能。arXiv 预印本 arXiv: 2310.12823。

张泽宇、薄晓和、马晨、李睿、陈旭、戴全宇、朱杰明、董振华和温继荣。2024. 基于大型语言模型的代理的记忆机制调查。arXiv 预印本 arXiv: 2404.13501。

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu 和 Gao Huang。2024. 驱逐: LIm 代理是体验式学习者。会议记录中

郑耀伟、张日昌、张俊浩、叶燕涵、罗哲燕、冯章驰和马永强。

2024. Llamafactory : 100+ 语言模型的统一高效微调。在计算语言学协会第 62 届年会 (第 3 卷: 系统演示) 的会议记录中, 泰国曼谷。计算语言学协会。

周王春书, 周雨辰, 江玲玲, 李龙, 吴佳龙, 王天南, 石秋, 张金田, 陈静, 吴瑞普, 王帅, et al.

2022

代理: 用于自治语言代理的开源框架。arXiv 预印本 arXiv: 2309.07870。

朱玉琦、乔硕飞、欧一欣、邓淑敏、张宁宇、吕世伟、沈悦、梁磊、顾金杰和陈华军。2024. Knowagent : 基于代理的llm知识增强规划。arXiv 预印本 arXiv: 2403.03101。

A 数据集详细信息

ScienceWorld ScienceWorld (Wang et al., 2022) 是一个基于文本的虚拟环境，为 AI 研究提供了一个测试平台，专门用于评估和改进 AI 系统的科学推理能力。研究人员可以使用此平台来评估 AI 代理在开放、复杂环境中的性能。

ScienceWorld 模拟标准小学科学课程中的任务，涵盖物质的状态变化、测量、电学、生命科学、植物生长、化学反应和遗传学等领域。代理部署在具体的交互式环境中，以理解和应用复杂的科学概念。ScienceWorld 中的任务涉及多个子目标，总体最终奖励是根据这些子目标的完成情况计算的。

ScienceWorld 的原始测试集包括看不见的任务变化。例如，在训练集中，任务可能涉及煮沸水，而在测试集中，任务可能是煮沸铅。遵循 Song et al. (2024b)，我们使用原始测试集来评估我们的元规划器在看不见的场景中的泛化能力，而原始验证集作为我们看得见的场景的测试集。

ALFWorld ALFWorld (Shridhar et al., 2020) 是需要代理探索房间并使用常识推理来执行任务的家务，例如“将铅笔放在桌子上”。该环境提供有关代理是否在给定步骤中成功完成任务的结果。原始的 ALFWorld 数据集包括可见和不可见的评估集。seen 集旨在评估分布内泛化，而具有新任务的 unseen 集测量代理的分布外泛化。

B 成功率

我们在表 7 中报告了实验的成功率。请注意，成功率的定义在两个任务之间有所不同。对于 ScienceWorld，原始论文没有提供成功率的具体定义。但是，根据官方环境，如果代理达到预定义的 latent 状态，即使奖励不完全是 1.0，也认为轨迹成功。对于 ALFWorld 来说，由于它只提供二进制最终奖励，因此成功率相当于平均最终奖励。插入 MPO 优化的元计划后

所有代理在这两项任务中都显示出一致且显著的成功率提高。

C Seed Meta 计划质量控制

高质量的种子元计划训练集对于初始化更有效的元计划器至关重要。因此，我们仔细控制 GPT-4o 生成的元计划的质量。我们确定了它生成的元计划的几个关键问题：(1) 它们通常包含过于详细的步骤或环境信息，这使得它们难以概括和优化；(2) 它们有时具有不适用于环境的作类型；(3) 他们没有遵守预定义的元计划格式。为了解决前两个问题，我们在 GPT-4o 生成过程中调整温度并重新总结了元计划。对于第三个问题，我们还提示 GPT-4o 从响应中提取格式正确的元计划。虽然需要人工验证来确保质量，但与 Zhu 等人的人工构建知识相比，这个过程所涉及的人力可以忽略不计。

(2024).

D 案例研究

在这里，我们提供了在插入通过两种不同方法优化的元计划后，ALFWorld 中同一任务的代理轨迹的详细比较：SFT 和 MPO。此比较展示了 MPO 如何提供更高质量的计划指导。案例如图 11 所示。本案例研究中使用的代理是 Llama-3.1-8B-Instruct。

在 ALFWorld 场景中，SFT 初始化的 meta planner 生成的 meta plan 错误地包含了“go to sidetable”的指令，误导智能体重复执行错误的计划“I can try to sidetable first”，导致计划幻觉。相比之下，MPO 优化的 meta 规划器会生成更高质量的 meta 计划：“go to the first pillow might be located where where the first pillow can be located.”该计划概述了一个抽象的、通用的任务完成策略，与特定的环境细节解耦，并正确地指导代理规划在环境中定位枕头，“我可以从 1 号扶手椅 1 开始逐一检查”。

e 我们工作中使用的提示符

E1 种子元计划集合的提示符

我们展示了 GPT-4o 根据任务说明生成种子元计划数据集的提示-

不带 Meta Plan 的模型	ScienceWorld ALFWorld 平均值											
	Seen 未见 Seen 未见											
未接受培训的座席												
GPT-4o (Achiam 等人, 2023 年)	X	60.0	56.0	78.6	83.6	69.6	✓	67.3	67.8	89.3	93.3	79.4
GPT-4o-mini (Achiam 等人, 2023 年)	X	49.1	42.7	32.1	41.0	41.2	✓	55.7	52.8	64.3	79.9	63.2
Llama-3.1-8B-Instruct (Dubey et al., 2024)	X	47.7	42.2	22.9	28.4	35.3	✓	56.5	55.5	50.0	52.2	53.6
Qwen2.5-7B-Instruct (Yang et al., 2024a)	X	38.5	38.8	71.4	75.4	56.0	✓	41.7	43.7	81.4	82.8	62.4
Llama-3.1-70B-Instruct (Dubey 等人, 2024)	X	72.6	70.2	78.6	73.9	73.8	✓	80.4	79.5	85.7	86.6	83.1
受过培训的代理												
Llama-3.1-8B-Instruct + SFT (Zeng 等人, 2023 年)	X	65.3	57.0	79.3	71.6	68.3	✓	70.2	65.9	80.7	81.3	74.5
Llama-3.1-8B-Instruct + ETO (Song 等人, 2024b)	X	81.3	74.1	77.1	76.4	77.2	✓	83.4	80.8	85.0	79.1	82.1

表 6: 在两个数据集上纳入 MPO 优化的元计划后, 不同代理的平均奖励比较。

不带 Meta Plan 的模型	ScienceWorld ALFWorld 平均值											
	Seen 未见 Seen 未见											
未接受培训的座席												
GPT-4o (Achiam 等人, 2023 年)	X	59.8	57.8	78.6	83.6	70.0	✓	61.3	65.9	89.3	93.3	77.5
GPT-4o-mini (Achiam 等人, 2023 年)	X	38.7	28.9	32.1	41.0	35.2	✓	41.2	41.2	64.3	79.9	56.7
Llama-3.1-8B-Instruct (Dubey et al., 2024)	X	25.8	25.6	22.9	28.4	25.7	✓	47.9	53.6	50.0	52.2	50.9
Qwen2.5-7B-Instruct (Yang et al., 2024a)	X	22.7	30.8	71.4	75.4	50.1	✓	32.0	33.2	81.4	82.8	57.4
Llama-3.1-70B-Instruct (Dubey 等人, 2024)	X	67.5	64.9	78.6	73.9	71.2	✓	71.7	69.7	85.7	86.6	78.4
受过培训的代理												
Llama-3.1-8B-Instruct + SFT (Zeng 等人, 2023 年)	X	59.3	64.9	79.3	71.6	68.8	✓	68.6	72.0	80.7	81.3	75.7
Llama-3.1-8B-Instruct + ETO (Song 等人, 2024b)	X	75.8	77.7	77.1	78.4	77.3	✓	80.9	78.7	85.0	79.1	80.9

表 7: 在两个数据集上纳入 MPO 优化的元计划后, 不同代理的成功率比较。对于 ALFWorld, 成功率相当于平均最终奖励。

tions 我们提供任务指令、环境信息和当前任务完成轨迹，然后提示 GPT-4o 提取一个包含环境先验的元计划，并可以指导任务完成过程。提示符如图 6 和图 7 所示。

E.2 评估提示

我们分别在图 8 和图 9 中显示了 ScienceWorld 和 ALFWorld 的指令提示。

E.3 GPT 自动评估提示

我们在图 10 中展示了提示，它使 GPT-4o 能够从正确性、可遵循性和标准化三个方面自动评估 MPO 优化的元计划的质量。正确性评估计划是否准确满足任务要求，可遵循性评估计划是否清晰、易于理解以及步骤是否合理，而标准化检查元计划是否遵循一致和标准化的格式。对于每个维度，GPT4o 被要求首先确定哪组计划更好并提供推理程序。最后，给出总体评估。

提示 ScienceWorld Meta Plan Collection

请为科学任务生成一个分步元计划： <task> 您是在环境中进行一些科学实验的有用助手。在环境中，有几个房间：厨房、铸造厂、车间、浴室、室外、客厅、卧室、温室、艺术工作室、走廊。{任务} </task>

您应该浏览环境并找到完成实验所需的项目。您可以一步传送到任何房间。环境中的所有容器都已经打开，您可以直接从容器中获取物品。

可用的作包括：打开 OBJ：打开容器 关闭 OBJ：关闭容器 激活 OBJ：激活设备 停用 OBJ：停用设备 将 OBJ 连接到 OBJ：连接电气元件 断开 OBJ：断开电气元件 使用 OBJ [在 OBJ 上]：使用设备/项目 环顾四周：描述当前房间 检查 OBJ：详细描述对象 查看 OBJ：描述容器的内容 阅读 OBJ：阅读笔记或书籍 将 OBJ 移动到 OBJ：将物体移动到容器上 拾取 OBJ：将物体移动到库存中 将 OBJ 倒入 OBJ：将液体倒入容器中 混合 OBJ：化学混合 容器 传送到 LOC：传送到特定房间 关注 OBJ：对任务对象发出信号意图 wait：任务对 10 个步骤无作 wait1：任务对步骤无作

以下是解决此任务的标准和详细过程： <conversation >
{conversation } </conversation >

您需要将抽象步骤总结为元计划，可用于将来解决类似的任务。

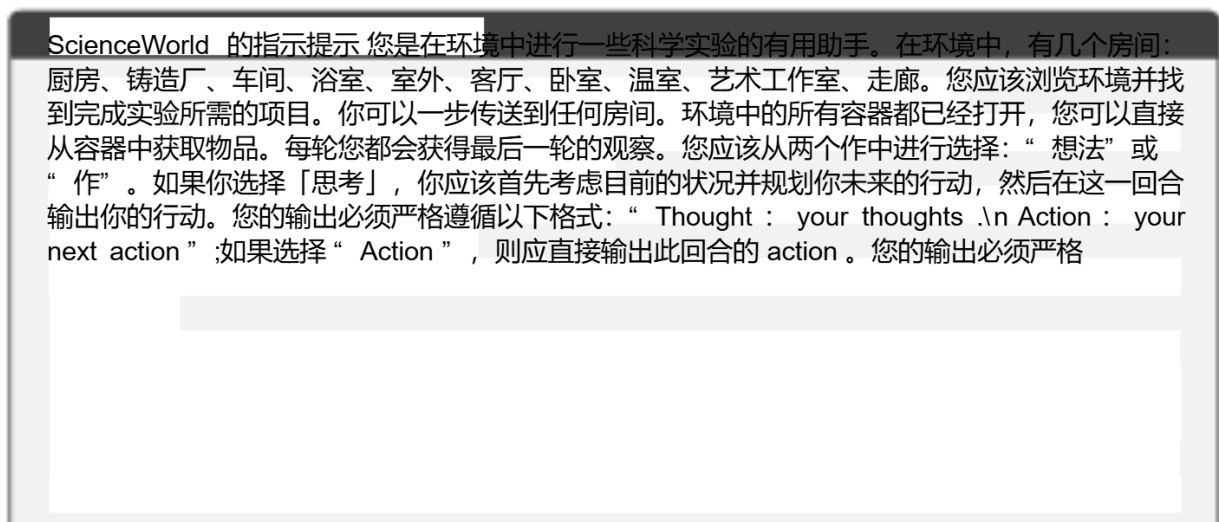
元计划应该是任务的经常重用例程。生成的元计划应按以下格式编写： <meta_plan> 第 1 步：...第 2 步：...

...
</meta_plan>

图 6: ScienceWorld Meta Plan Collection 的提示。



图 7: ALFWorld 元计划集合的提示。



您的输出必须严格遵循以下格式：“ Action : your next action ”。请记住，每个响应只能输出一个“ Action : ”。

可用的作包括：打开 OBJ: 打开容器 关闭 OBJ: 关闭容器 激活 OBJ: 激活设备 停用 OBJ: 停用设备 将 OBJ 连接到 OBJ: 连接电气元件 断开 OBJ: 断开电气元件 使用 OBJ [在 OBJ 上]: 使用设备/项目 环顾四周: 描述当前房间 检查 OBJ: 详细描述对象 查看 OBJ: 描述容器的内容 阅读 OBJ: 阅读笔记或书籍 将 OBJ 移动到 OBJ: 将物体移动到容器上 拾取 OBJ: 将物体移动到库存中 将 OBJ 倒入 OBJ: 将液体倒入容器中 混合 OBJ: 化学混合 容器 传送到 LOC: 传送到特定房间 关注 OBJ: 任务对象上的信号意图 wait : 10 个步骤的任务无作 wait1 : 步骤的任务无作 - 下面是一个示例。{示例} - - - 现在，轮到你了，任务来了。{task_instruction }

这个元计划可能对你完成任务有所帮助：{meta_plan }

图 8: ScienceWorld 的指令提示。

ALFWorld 的指令提示

与住户互动以解决任务。想象一下，您是家庭环境中的智能代理，您的目标是执行作以完成任务目标。在交互开始时，您将获得当前环境的详细说明和您要完成的目标。

每轮您都会获得最后一轮的观察。您应该从两个作中进行选择：“想法”或“作”。如果你选择「思考」，你应该首先考虑目前的状况并规划你未来的行动，然后在这一回合输出你的行动。您的输出必须严格遵循以下格式：“ Thought : your thoughts \n Action : your next action ”；如果选择“ Action ”，则应直接输出此回合的 action 。您的输出必须严格遵循以下格式：“ Action : your next action ”。可用的作包括：

1. 前往 RECEP
2. 从 Recep 获取 OBJ
3. 将 OBJ 放入/打开 recep
4. 打开 RECEP
5. 关闭接收
6. 切换 OBJ RECEP
7. 使用 RECEP清洁 OBJ
8. 用 Recep 加热 OBJ
9. 使用 recep 冷却 obj 可用的作是：其中 obj 和 recep 对应于对象和容器。
每次轮次后，环境都会根据你计划接下来的几个步骤立即给你反馈。如果环境输出“ Nothing happened ”，则表示上一个作无效，您应该尝试更多选项。
提醒： 1. 必须从给定的可用作中选择作。除提供的可用作外，任何作都将被视为非法。

2. 必要时思考，尽量在这个过程中更直接地行动。

下面是一个示例。
{示例}
--- 现在，轮到你了，任务来了。
{task_instruction }

这个元计划可能对你完成任务有所帮助：{meta_plan }

图 9：ALFWorld 的指令提示。

GPT 自动评估的指示提示

请作为专业的教学评估员，评估以下两套元计划。

任务描述：{task}

DPO 计划：

{dpo}

SFT计划：

{sft}

请从以下三个维度比较这两套元计划：1. 正确性 - 元计划是否准确满足任务要求？

2. 可遵循性 - 元计划是否清晰、易于理解，步骤是否合理？ 3. 标准化 - 元计划是否遵循一致和标准化的格式？

对于每个维度，请指出哪个元计划更好并提供理由。最后，提供总体评估。请以JSON格式输出结果，包括以下字段：{ "correctness_better" : " dpo" /" sft" /" tie" ,
" correctness_reason" : " reason" , " followability_better " : " dpo" /" sft" /" tie" ,
" followability_reason " : " reason" , " standardization_better " : " dpo" /" sft" /
" tie" , " standardization_reason " : " reason" , " overall_better " : " dpo" /" sft" /
" tie" }

图 10：GPT 自动评估的指令提示。

SFT

任务指令：你在房间的中间。快速环顾四周，你会看到一个扶手椅 1、一个柜子 4、一个柜子 3、一个柜子 2、一个柜子 1、一个抽屉 5、一个抽屉 4、一个抽屉 3、一个抽屉 2、一个抽屉 1、一个梳妆台 1、一个垃圾桶 1、一个保险箱 1、一个架子 12、一个架子 11、一个架子 10、一个架子 9、一个搁板 8、一个搁板 7、一个搁板 6、一个搁板 5、一个搁板 4、一个搁板 3、一个搁板 2、一个搁板 1、一个边桌 1 和一个沙发 1。你的任务是：找到两个枕头，把它们放在沙发上！

糟糕的元计划

Meta 计划：第 1 步：去沙发。第 2 步：前往 sidetable。第 3 步：从侧桌上拿起枕头。第 4 步：前往 SOFA。第 5 步：将枕头放在沙发上。第 6 步：前往 sidetable。第 7 步：从侧桌上拿另一个枕头。第 8 步：前往沙发。第 9 步：将第二个枕头放入/放在沙发上。

想法：要解决任务，我需要找到两个枕头并将它们放在沙发上。元计划建议我应该先去沙发，然后再去边桌。

动作：去沙发 观察：什么都没发生。

思路：看来“go to sofa”这个动作是无效的。我可以尝试再次去沙发，或者我可以尝试先去边桌。

作：转到 sidetable 观察：什么都没有发生。

思考：看来“go to sidetable”这个动作也是无效的。我可以尝试再次去沙发，或者我可以尝试去柜子或抽屉。

动作：进入机柜 1 观察：机柜 1 关闭。

想：现在我到了柜子 1 前面，可以试着打开它，看看里面有没有枕头。

作：打开机柜 1 观察：您打开机柜 1。机柜 1 已打开。在其中，您会看到一个纸巾盒 1。

想：看来柜子里没有枕头 1。我可以尝试再次去边桌看看有没有枕头。

作：转到 sidetable 观察：什么都没有发生。

MPO

任务指令：你在房间的中间。快速环顾四周，你会看到一个扶手椅 1、一个柜子 4、一个柜子 3、一个柜子 2、一个柜子 1、一个抽屉 5、一个抽屉 4、一个抽屉 3、一个抽屉 2、一个抽屉 1、一个梳妆台 1、一个垃圾桶 1、一个保险箱 1、一个架子 12、一个架子 11、一个架子 10、一个架子 9、一个搁板 8、一个搁板 7、一个搁板 6、一个搁板 5、一个搁板 4、一个搁板 3、一个搁板 2、一个搁板 1、一个边桌 1 和一个沙发 1。你的任务是：找到两个枕头，把它们放在沙发上！

好的元计划

元计划：第 1 步：前往第一个枕头可能所在的位置。第 2 步：拿第一个枕头。第 3 步：前往沙发所在的地方。第 4 步：将第一个枕头放入/放在沙发上。第 5 步：前往第二个枕头可能所在的位置。第 6 步：拿第二个枕头。第 7 步：回到沙发上。第 8 步：将第二个枕头放入/放在沙发上。

想法：枕头更容易出现在扶手椅（1）、橱柜（1-4）、抽屉（1-5）、梳妆台（1）、架子（1-12）、沙发（1）中。我可以从 armchair 1 开始逐一检查。

动作：去扶手椅 1 观察：在扶手椅 1 上，你会看到一个枕头 2、一个枕头 1 和一个遥控器 1。

思考：现在我找到了第一个枕头（1）。接下来，我需要接受它。我稍后可以在 1 号扶手椅中找到第二个枕头（2）。

动作：从扶手椅 1 中取出枕头 1 观察：从扶手椅 1 上拿起枕头 1。

想：现在我拿第一个枕头（1）。接下来，我需要把它放在沙发上。我可以直接去沙发。

动作：去沙发 1 观察：在沙发 1 上，你什么也看不到。

想：现在我去沙发上。接下来，我需要将第一个枕头放入/放在沙发上。我可以直接把第一个枕头放在沙发上。

动作：将枕头 1 放入/放在沙发 1 上 观察：您将枕头 1 放入/放在沙发 1 上。

想：现在我把第一个枕头放在沙发上。接下来，我需要找到第二个枕头。我可以直接去 1 号扶手椅

想法：看来“ go to sidetable ” 这个动作还是无效的。我可以尝试再次转到 sofa 以启动工作流程。 动作：去沙发 观察：什么都没发生。 思路：看来“ go to sofa ” 这个动作还是无效的。我可以尝试改为转到抽屉，因为 workflow 没有指定要转到哪个抽屉。 作：转到抽屉 1 观察：抽屉 1 已关闭。 思考：看来柜子里没有枕头 2。我可以尝试转到不同的机柜或位置。 动作：前往 3 号机柜 观察：3 号机柜关闭。 想：现在我已经到了柜子 3 的前面，可以试着打开它，看看里面有没有枕头。 动作：打开柜子 3 观察：你打开柜子 3。机柜 3 已打开。在其中，您会看到一个盒子 3 和一个纸巾盒 2。 奖励： 0.0	动作：去扶手椅 1 观察：在扶手椅 1 上，你会看到一个枕头 2 和一个遥控器 1。 想法：现在我找到了第二个枕头（2）。 接下来，我需要接受它。 动作：从扶手椅 2 中取出枕头 1 观察：您从扶手椅 2 上拿起枕头 1。 思考：现在我拿第二个枕头（2）。 接下来，我需要把它放在沙发上。动作：去沙发 1 观察：沙发 1 上，你看到一个枕头 1. 想：现在我去沙发上。接下来，我需要将第二个枕头放入/放在沙发上。我可以直接将第二个枕头放入/放在沙发上。 动作：将枕头 2 放入/放在沙发上 1 观察：您将枕头 2 放入/放在沙发上 1. 奖励： 1.0
--	--

图 11：ALFWorld 的案例研究。